

1. Arrays (Datenfelder)

Arrays werden genutzt, um eine feste Anzahl gleichartiger Werte unter einem gemeinsamen Namen zu speichern.

- **Struktur:** Arrays sind in der Regel **eindimensional** und bestehen aus einem **Index** (die Position) und dem dazugehörigen **Wert**.
- **Indizierung:** Der Index beginnt in fast allen Prüfungsaufgaben bei **0**. Ein Array mit 24 Feldern hat somit die Indizes 0 bis 23.
- **Deklaration:** Die Erstellung erfolgt meist über Befehle wie `int[] name = new int[größe]`.
- **Eigenschaften:** Mit der Methode `.Length` oder `Length()` kann die Anzahl der Felder eines Arrays ermittelt werden.

2. Pseudocode-Syntax

In den Prüfungsanlagen wird oft eine Syntax-Hilfe bereitgestellt, die folgende Elemente umfasst:

- **Schleifen (Iterationen):**
 - **for-Schleife:** Ideal zum Durchlaufen von Arrays, wenn die Anzahl der Durchläufe bekannt ist.
 - **while-Schleife:** Kopfgesteuerte Schleife, die läuft, solange eine Bedingung wahr ist.
 - **do...while-Schleife:** Fußgesteuerte Schleife, die mindestens einmal ausgeführt wird.
- **Auswahlweisungen (Verzweigungen):**
 - **if...else:** Prüfung von Bedingungen.
 - **Logische Operatoren:** `&&` (UND-Verknüpfung), `||` (ODER-Verknüpfung).
- **Operatoren:** Neben Standardrechenarten sind die **Ganzzahldivision** (`/`) und der **Modulo-Operator** (`%`) wichtig, um Reste zu berechnen oder Stellen aus IDs zu extrahieren.

3. Typische Algorithmen (Prüfungs-Klassiker)

In den Aufgaben musst du oft bestehende Entwürfe vervollständigen:

- **Mittelwertberechnung:** Summiere alle Werte des Arrays in einer Schleife auf und teile das Ergebnis am Ende durch die Anzahl der Elemente.
- **Maximum-Suche:** Initialisiere eine Variable `max = 0` und vergleiche in einer Schleife jedes Array-Element mit diesem Wert. Ist das Element größer, aktualisiere `max`.
- **Schwellenwert-Überwachung:** Zähle, wie oft ein Wert im Array einen vorgegebenen Grenzwert (z. B. 80 % CPU-Last) überschreitet, und gib ggf. eine Meldung aus.

- **Sortiervverfahren (Bubblesort):** Nutze zwei verschachtelte Schleifen, um benachbarte Elemente zu vergleichen und bei Bedarf mithilfe einer Hilfsvariable (`temp`) zu tauschen.
- **Daten-Transformation:** Berechne Mittelwerte aus Dreiergruppen eines Arrays und schreibe die Ergebnisse in ein neues Array (Glättung von Werten).

4. Fehleranalyse und Debugging

Oft musst du Fehler in vorgegebenem Code finden:

- **IndexOutOfRangeException:** Ein sehr häufiger Fehler. Er tritt auf, wenn die Schleifenbedingung falsch gesetzt ist (z. B. `i <= array.Length` statt `i < array.Length`), wodurch auf ein nicht existierendes Feld zugegriffen wird.
- **Syntax- vs. Semantikfehler:**
 - **Syntaxfehler:** Das Programm ist formal falsch geschrieben (z. B. Tippfehler).
 - **Semantikfehler:** Das Programm läuft, aber die Logik ist falsch und führt zu falschen Ergebnissen.
- **Trace-Tabellen:** Zur Analyse musst du den Code Schritt für Schritt "durchspielen" und die Werte der Variablen (`i`, `max`, `summe`) nach jedem Schleifendurchlauf in eine Tabelle eintragen.

5. Theoretische Grundlagen

- **Vorteile von Pseudocode:** Er ist unabhängig von konkreten Programmiersprachen und konzentriert sich rein auf die **Logik des Algorithmus**.
- **Objektorientierung:** Begriffe wie **Datenkapselung** (Schutz von Daten) und **Vererbung** (Weitergabe von Eigenschaften) sollten erklärt werden können.
- **Testverfahren:** Unterschied zwischen **White-Box-Tests** (Kenntnis des Quellcodes) und **Black-Box-Tests** (Prüfung nur von Ein- und Ausgabewerten).